Lab 1: Agile Development and Git

Developing AI-enabled Systems

Alexander Forsanker, Department of Computing
Jönköping AI Lab (JAIL) @ Jönköping University

Goal: Get familiar with Python, Git, GitHub/GitHub actions, and a simple ML classifier. The goal is simply to practice building a workflow, not build or optimize an ML model.

1 Purpose and learning outcomes

In this lab you will use Git, GitHub, selected Scrum concepts, user stories, pull requests (PRs), and continuous integration (CI) while working with a small AI-enabled application.

After completing the lab, you should be able to:

1. Translate a requirement into a user story with clear acceptance criteria.
2. Plan and track a short iteration using a simplified, individual Scrum-inspired workflow.
3. Use Git commits and feature branches to isolate changes.
4. Open, review, and merge your own GitHub pull request.
5. Configure GitHub Actions to run automated tests on pushes and pull requests.
6. Describe what tests can and cannot validate in an AI-enabled system.

2 Prerequisites and provided code

You need Python 3.9+, Git, a GitHub account, and a code editor such as VS Code. You will create your own GitHub repository. The provided folder contains:

- `app.py`: Flask web interface and JSON `/predict` endpoint.
- `model.py`: scikit-learn classifier trained using the built-in Iris dataset.
- `tests/test_app.py`: two *working example tests* using pytest and Flask's test client.
- `requirements.txt`, `README.md`, and `.gitignore`.

The application receives four flower measurements and predicts an Iris species. The classifier is already implemented. You will run the provided tests, configure CI and, if you wish, add tests of your own. No GitHub Actions workflow is provided because creating one is part of the exercise.

3 Part A: Explore the application

Extract the archive and open the `lab_1` directory in a terminal. Create a virtual environment, install the dependencies and start the application:

```
python -m venv .venv
# macOS/Linux:
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

Visit http://127.0.0.1:5000, press *Predict* and try changing the measurements. Then inspect `model.py` and the `/predict` endpoint in `app.py`. Stop the server with `Ctrl+C` when you are finished.

Reflect about this: What are the input and output of the model? What happens when the endpoint receives missing, malformed or out-of-range measurements? Write down your notes in a markdown file called **reflections.md**.

4 Part B: Plan a small iteration

Scrum is designed for teams, but some principles are useful when planning individual work: maintaining a backlog, setting a sprint goal, checking progress and reflecting on the result. For this lab, simulate one short iteration. You do *not* need to assign Scrum roles or organize meetings.

Create **two GitHub Issues** using the user-story format:

As a [type of user], I want [capability], so that [benefit].

Examples/Suggested stories:

1. **Automated verification.** As a developer, I want the existing API tests to run on GitHub whenever I push or open a pull request, so that regressions are detected early.

2. **Additional input validation coverage.** As a developer, I want tests for further invalid input cases, so that I can be more confident the API returns useful errors. This issue can remain in the backlog if you choose not to write more tests.

For each issue, write **at least two acceptance criteria**, preferably in *Given/When/Then* form. Record a simple board with **To do**, **In progress** and **Done** (using GitHub Projects, or a checklist or table in an Issue). Write a **one-sentence sprint goal**.

5 Part C: Create your GitHub repository

Create a new GitHub repository, for example `iris-predictor-lab`. Do not initialize it with a README, since the lab_1 directory already has one. In the main directory, and run:

```
git init
git branch -M main
git add .
git commit -m "Initial Iris predictor app"
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git push -u origin main
```

Replace `USERNAME/REPOSITORY` with your own path. You will find the instructions on how to add the remote to your local repo once you have created the repo on GitHub. Do not use GitHub Desktop since you lose access to useful commands.

Create a feature branch for your work:

```
git switch -c feature/automated-tests
```

or

```
git checkout -b feature/automated-tests
```

Do the following changes on your feature branch, not on `main`.

6 Part D: Run and understand the supplied tests

Open `tests/test_app.py`. It contains **two complete tests**:

1. A valid POST request to `/predict` verifies HTTP 200 and an allowed `species` label.
2. A request containing only two measurements verifies HTTP 400 and a JSON `error` field.

Run them locally without starting the Flask server:

```
python -m pytest -v
```

Inspect: How does `app.test_client()` simulate HTTP requests? What do the assertions check? Why is a test that checks the API contract often preferable to one tied to a model's precise internal predictions?

Optional challenge: add one or more tests

You are encouraged to implement your own test, but the two provided tests are sufficient for completing the core CI exercise. Ideas include verifying that `GET /` returns HTTP 200, that missing measurements return HTTP 400, or that a nonnumeric measurement is rejected.

For example, you could begin with:

```
def test_missing_measurements():
    client = app.test_client()
    response = client.post("/predict", json={})
    # Write your own assertions here.
```

Run pytest again after any change. If you implement extra tests, update the second Issue with the tests you covered.

7 Part E: Configure GitHub Actions CI

GitHub Actions can execute the tests automatically when changes are pushed and when a PR is opened. Create `.github/workflows/tests.yml`. Your workflow should:

1. Trigger on pushes and pull requests targeting `main` (you may also run it on pushes to feature branches).
2. Run on an Ubuntu runner and check out the repository.
3. Set up Python 3.11, then install `requirements.txt`.
4. Run `python -m pytest -v`.

Hints: Consult the official GitHub Actions documentation. Look for the `push` and `pull_request` triggers, `actions/checkout`, and `actions/setup-python`. Commands to install dependencies and run tests go in `run:` steps. YAML indentation matters. *Do not* include secrets in the workflow file.

Check that the tests pass locally, then commit and push:

```
git add .github/workflows tests
git commit -m "Configure CI for existing API tests"
git push -u origin feature/automated-tests
```

If Git reports there is nothing to commit in `tests/`, that is expected when you have left the provided tests unchanged: they were committed during initial setup. The new workflow is the main feature of this branch.

8 Part F: Open, review and merge your own PR

On GitHub, open a pull request from `feature/automated-tests` into `main`. Link your first Issue in the PR description (for example, `Closes #1`), summarize the change, and state how you tested it.

You are working individually, so **no other reviewer is needed**. Instead, inspect the *Files changed* tab and leave a short **self-review comment** noting one strength or limitation of your work. Check the *Actions* or *Checks* tab and confirm that the workflow passed. Then merge the PR and verify that the Issue is closed (or close it yourself) and marked **Done**.

9 Part G: Observe a failing pipeline (optional)

To see how CI catches failures, switch to `main`, pull the newly merged changes and make a new branch:

```
git switch main
git pull
git switch -c experiment/failing-test
```

Change the HTTP 200 assertion in the valid-prediction test to expect HTTP 201. Verify that pytest fails locally. Commit and push the change, then open a **draft** PR into `main` to observe the failing GitHub Actions check. Restore the correct assertion, push again, and confirm the test passes. Close the draft PR without merging it. If you wish, delete the experiment branch afterwards.

10 Conclusion and reflection

The lab is **optional** and carried out **individually**; no formal submission is required for this version. When finished, you should have a repository containing the starter app (lab_1 directory), GitHub Issues with acceptance criteria, at least one feature-branch PR, and a working CI workflow.

Write a brief personal reflection (about 100–200 words, for your own learning): How did user stories and a sprint goal help you organize the work? What did the PR and automated workflow add, even without collaborators? What kinds of failures can these tests detect, and what aspects of real-world ML reliability do they leave untested?